

```
1 using CustomProjectRPG.Core.Utilities;
2 using CustomProjectRPG.Interfaces;
3 using System;
4 using System.Collections.Generic;
5 using System.Linq;
6 using System.Text;
7 using System.Text.Json;
8 using System.Text.Json.Serialization;
9 using System.Threading.Tasks;
10 using Enum = CustomProjectRPG.Entities.Enum.Enum;
11
12 namespace CustomProjectRPG
13 {
14
15
16
17     public class CorruptionComponent : ISerializeJSON<CorruptionComponent>, ICorruption ➤
18     {
19         private float _corruptionPercentage;
20         private int _warningCount;
21         private Enum.EntityType _entityType;
22
23         public float CorruptionPercentage => _corruptionPercentage;
24         public int WarningCount => _warningCount;
25
26
27         /// Initializes a new CorruptionComponent for a specific entity type with an optional initial corruption value. ➤
28
29         public CorruptionComponent(Enum.EntityType entityType, float initialCorruption = 0f) ➤
30         {
31             _entityType = entityType;
32             _corruptionPercentage = Math.Clamp(initialCorruption, 0f, 100f); ➤
33             if (_entityType == Enum.EntityType.Player)
34             {
35                 _warningCount = 0;
36             }
37         }
38
39
40         /// Increases corruption by the specified amount, capping at 100%.
41
42         public void IncreaseCorruption(float amount)
43         {
44             if (amount < 0) throw new ArgumentException("Amount must be non-negative.", nameof(amount)); ➤
```

```
45         _corruptionPercentage = Math.Min(_corruptionPercentage +  
46             amount, 100f);  
47     }  
48  
49     /// Decreases corruption by the specified amount, flooring at 0%.  
50  
51     public void DecreaseCorruption(float amount)  
52     {  
53         if (amount < 0) throw new ArgumentException("Amount must be  
54             non-negative.", nameof(amount));  
55         _corruptionPercentage = Math.Max(_corruptionPercentage -  
56             amount, 0f);  
57     }  
58  
59     /// Returns the corruption state (used primarily for beasts).  
60  
61     public Enum.CorruptionState GetCorruptionState()  
62     {  
63         if (_corruptionPercentage == 0f) return  
64             Enum.CorruptionState.Normal;  
65         if (_corruptionPercentage == 100f) return  
66             Enum.CorruptionState.Full;  
67         return Enum.CorruptionState.Partial;  
68     }  
69  
70     /// Checks if the player should receive a corruption warning from  
71     the daughter.  
72  
73     public bool ShouldShowWarning()  
74     {  
75         return _entityType == Enum.EntityType.Player &&  
76             _corruptionPercentage >= 77f && _warningCount < 3;  
77     }  
78  
79     /// Increments the warning count for the player.  
80  
81     public void IncrementWarningCount()  
82     {  
83         if (_entityType == Enum.EntityType.Player)  
84         {  
85             _warningCount++;  
86         }  
87     }
```

```
87     /// Checks if the player's corruption triggers the bad ending.
88
89     public bool ShouldTriggerBadEnding()
90     {
91         return _entityType == Enum.EntityType.Player &&           ↗
92             _corruptionPercentage >= 77f && _warningCount >= 3;
93     }
94
95     /// Checks if an NPC's dialogue is corrupted (≥50% corruption).
96
97     public bool IsDialogueCorrupted()
98     {
99         return _entityType == Enum.EntityType.NPC &&             ↗
100             _corruptionPercentage >= 50f;
101     }
102
103     /// Checks if an NPC's house is a placeholder (≥60% corruption).
104
105     public bool IsHousePlaceholder()
106     {
107         return _entityType == Enum.EntityType.NPC &&             ↗
108             _corruptionPercentage >= 60f;
109     }
110
111     /// Checks if an NPC stops responding (≥80% corruption).
112
113     public bool IsNoResponse()
114     {
115         return _entityType == Enum.EntityType.NPC &&             ↗
116             _corruptionPercentage >= 80f;
117     }
118
119     /// Updates corruption at the start of a new loop based on entity ↗
120     type.
121
122     public void OnNewLoop()
123     {
124         if (_entityType == Enum.EntityType.Player)
125         {
126             IncreaseCorruption(10f); // Player: +10% per loop
127         }
128         else if (_entityType == Enum.EntityType.NPC)
129         {
130             Random rand = new Random();
131             float increase = rand.Next(11, 18); // NPC: +11% to 17% ↗
```

```
        per loop
131         IncreaseCorruption(increase);
132     }
133     // Beasts: No change (set at spawn)
134 }
135
136
137     /// Calculates the stat multiplier for the player based on corruption level.
138
139     public float GetStatMultiplier()
140     {
141         if (_entityType != Enum.EntityType.Player) return 1.0f;
142         if (_corruptionPercentage < 50f) return 1.0f;
143         int steps = (int)((_corruptionPercentage - 50f) / 10f);
144         float multiplier = 0.8f - steps * 0.1f; // 20% decrease at 50%, then 10% per 10%
145         return Math.Max(multiplier, 0f);
146     }
147
148
149     public event Action<float> CorruptionChanged;
150     private void OnCorruptionChanged() => CorruptionChanged?.Invoke(_corruptionPercentage);
151
152     /// Serializes the corruption state to JSON.
153
154     public string Serialize()
155     {
156         var data = new CorruptionData
157         {
158             CorruptionPercentage = _corruptionPercentage,
159             WarningCount = _warningCount
160         };
161         return SerializeJSON.ToJson(data);
162     }
163
164
165     /// Deserializes the corruption state from JSON.
166
167     public void Deserialize(string json)
168     {
169         var data = SerializeJSON.FromJson<CorruptionData>(json);
170         _corruptionPercentage = Math.Clamp(data.CorruptionPercentage, 0f, 100f);
171         if (_entityType == Enum.EntityType.Player)
172         {
173             _warningCount = data.WarningCount;
174         }
175     }
176 }
```

```
175     }
176
177     private class CorruptionData
178     {
179         [JsonPropertyName("corruptionPercentage")]
180         public float CorruptionPercentage { get; set; }
181
182         [JsonPropertyName("warningCount")]
183         public int WarningCount { get; set; }
184     }
185
186     }
187 }
188
```